

PRG1 – Test écrit 1

Exercice 1 (20 points)

Qu'affichent les codes suivants ?

1.1	<code>cout << 3 + 2 * 2 / 3;</code>	
1.5	<code>cout << boolalpha << (2 > 2 != 2);</code>	
1.7	<code>int i = 1; cout << (i-- and ++i ? ++i : i--) ;</code>	
1.8	<code>cout << ("Hello, " , "World !");</code>	
1.9	<code>string str; for(char c = 'A' ; c <= 'C'; ++c) str = c + str + c; cout << str;</code>	
1.10	<code>string str; for(char c = 'A' ; c <= 'C'; ++c) str = str + c + str; cout << str;</code>	
1.11	<code>int cnt = 0; for(int i = 0; i <= 3; ++i) for(int j = 2; j < 5; ++j) for(int k = 1; k < 10; k*=2) cnt++; cout << cnt;</code>	

1.13	<pre> int i = 3; while(true) { if(++i % 2 == 0) continue; if(++i > 10) break; cout << i << " "; } </pre>	
1.14	<pre> for (int i = 'C'; i >= 'A'; --i) { switch (i) { case 'B': cout << 2; break; default: cout << 1; case 'C': cout << 3; continue; } cout << 0; } </pre>	
1.15	<pre> int a = 4; int& b = a; int c = b; const int& d = c; const int e = c; a -= 1; b *= 2; c /= 2; cout << a << " " << b << " " << c << " " << d << " " << e ; </pre>	
1.16	<pre> int i = 1; do { ++i; cout << (i % 2 ? "A" : "B"); } while (i<5); </pre>	

1.17	<pre>for(int i = 1; i <= 10; --i) { cout << (i *= 3) << " "; }</pre>	3 6 15
------	--	--------

Exercise 2 (5 points)

De quels types sont les expressions suivantes ?

2.3	cin or cout	
2.4	'A' < 'B' ? .1 : .2	
2.5	(2.f + 3.L , 'A' + 'B')	

Exercice 3 – Nombre hautement composé

En arithmétique, un nombre *hautement composé* (HC) est un entier strictement positif qui a strictement plus de diviseurs que n'en ont les nombres qui le précèdent.

Par exemple, le nombre 6 a 4 diviseurs - 1, 2, 3, et 6 – ce qui est strictement plus que tout nombre plus petit que 6, et donc le nombre 6 est HC. Ainsi, les 10 premiers nombres HC sont 1, 2, 4, 6, 12, 24, 36, 48, 60, et 120. Ils ont respectivement 1, 2, 3, 4, 6, 8, 9, 10, 12 et 16 diviseurs.

Écrivez un programme complet qui demande à l'utilisateur « combien de nombres HC ? », et qui affiche les n premiers nombres HC en fonction du nombre n demandé.

Pour rendre votre code plus lisible, écrivez une fonction `nbDiviseurs` qui retourne le nombre de diviseurs du nombre entier strictement positif passé en paramètre. Cette fonction doit être déclarée avant la fonction `main` et définie après elle. On suppose que l'utilisateur fournit une entrée correcte et il n'est donc pas demandé de la valider.

Exemple d'exécution, avec l'entrée utilisateur en vert :

```
Combien de nombres HC ? 15
1 2 4 6 12 24 36 48 60 120 180 240 360 720 840
```